

ABSTRACT

Twitter is a major platform for sharing opinions, reactions, and experiences in real time. The large volume of textual data generated on social media makes manual analysis difficult. This project presents a machine-learning based Twitter Sentiment Analysis system that automatically classifies tweets into Positive, Negative, and Neutral sentiment categories. The project uses a labelled dataset containing 27,481 tweets. The textual data is preprocessed to reduce noise and prepare it for machine learning. TF-IDF (Term Frequency–Inverse Document Frequency) is used to transform the cleaned tweets into numerical feature vectors. A Linear Support Vector Classification (LinearSVC) model is then trained using an 80:20 stratified train-test split. The trained model achieves an accuracy of 65.99%, with precision of 66.38%, recall of 65.99%, and F1-score of 66.07%. The results demonstrate that traditional machine-learning techniques can provide a useful baseline for automated sentiment classification of social-media text.

1. INTRODUCTION

Sentiment analysis, also known as opinion mining, is the process of identifying the emotional orientation expressed in text. It is widely used to understand customer opinions, public reactions, product feedback, and social-media discussions. Twitter provides a large and continuously changing source of short text messages, making it suitable for sentiment analysis.

In this project, machine learning is applied to classify tweets into three sentiment classes: Positive, Negative, and Neutral. Since machine-learning algorithms cannot directly process raw text, the tweets are first cleaned and converted into numerical representations using TF-IDF. A LinearSVC classifier is subsequently used to learn patterns associated with each sentiment class.

2. PROBLEM STATEMENT

Manual examination of thousands of tweets is time-consuming and subjective. There is therefore a need for an automated system that can process large quantities of Twitter text and classify the expressed sentiment efficiently. The proposed system addresses this problem by combining text preprocessing, TF-IDF feature extraction, and supervised machine learning.

3. OBJECTIVES

- To preprocess and clean Twitter text for machine-learning analysis.
- To transform textual data into numerical features using TF-IDF.
- To train a supervised sentiment classification model using LinearSVC.
- To classify tweets as Positive, Negative, or Neutral.
- To evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- To demonstrate a practical machine-learning workflow for social-media sentiment analysis.

4. EXISTING SYSTEM

Traditional sentiment analysis may involve manual reading of tweets or rule-based methods using predefined keywords and sentiment dictionaries. Such approaches can be difficult to maintain and may fail when tweets contain informal language, abbreviations, context-dependent expressions, or mixed sentiment. Manual analysis also becomes impractical as the dataset grows.

5. PROPOSED SYSTEM

The proposed system automatically processes a labelled collection of tweets and learns sentiment patterns from the training data. The workflow consists of data loading, preprocessing, train-test splitting, TF-IDF feature extraction, LinearSVC model training, prediction, and performance evaluation.

6. SYSTEM REQUIREMENTS

6.1 Software Requirements

- Python 3.x
- Jupyter Notebook / JupyterLab
- Pandas
- NumPy
- Scikit-learn
- Matplotlib / visualization libraries

6.2 Hardware Requirements

- Minimum 4 GB RAM (8 GB recommended)
- Sufficient storage for the dataset and notebook
- Internet connection when obtaining the dataset or installing packages

7. DATASET DESCRIPTION

The dataset used in the notebook contains 27,481 tweet records. Each record contains textual information and an associated sentiment label. The target sentiment categories are Positive, Negative, and Neutral. The dataset is used as labelled data for supervised learning.

Dataset Attribute	Value
Number of tweets	27,481
Task	Multi-class text classification
Sentiment classes	Positive, Negative, Neutral
Training/testing split	80% / 20%
Split strategy	Stratified

First 5 rows:				
	textID	text	selected_text	sentiment
0	cb774db0d1	I'd have responded, if I were going	I'd have responded, if I were going	neutral
1	549e992a42	Sooo SAD I will miss you here in San Diego!!!	Sooo SAD	negative
2	088c60f138	my boss is bullying me...	bullying me	negative
3	9642c003ef	what interview! leave me alone	leave me alone	negative
4	358bd9e861	Sons of ****, why couldn't they put them on the releases we already bought	Sons of ****,	negative

8. METHODOLOGY

The methodology follows a standard supervised machine-learning pipeline. Raw tweets are first cleaned to remove irrelevant textual noise. The cleaned text is separated into training and testing data. TF-IDF converts the text into weighted numerical vectors. LinearSVC is trained on the training vectors and then used to predict the sentiment of unseen test tweets. The predictions are evaluated against the true labels.

9. DATA PREPROCESSING

Twitter text contains URLs, punctuation, special symbols, repeated characters, user mentions, hashtags, and other elements that may introduce noise. The preprocessing stage prepares the text for feature extraction. The notebook's preprocessing pipeline cleans the tweet content before applying TF-IDF.

- Remove or normalize unwanted textual elements.
- Prepare the tweet text consistently for feature extraction.
- Reduce noise while preserving information useful for sentiment classification.

```
Missing values per column:
textID          0
text            1
selected_text    1
sentiment        0
dtype: int64
```

Detecting the Text and Sentiment Columns

Rather than assuming column names, we inspect the dataset directly. This dataset (a well-known Twitter sentiment extraction dataset) contains:

- `textID` — a unique identifier for each tweet (not useful for modeling)
- `text` — the **raw tweet text** → this is our text/comment column
- `selected_text` — a sub-span of the tweet highlighting the sentiment-bearing phrase (not needed for whole-tweet sentiment classification, so it is not used)
- `sentiment` — the **sentiment label** → this is our target column

10. TF-IDF FEATURE EXTRACTION

TF-IDF stands for Term Frequency–Inverse Document Frequency. It converts documents into numerical feature vectors while assigning greater importance to terms that are frequent in a particular document but less common across the complete collection.

The basic TF-IDF representation can be expressed as:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

Here, $\text{TF}(t,d)$ represents the frequency of term t in document d , while IDF gives lower weight to terms that occur in many documents. This representation allows the LinearSVC algorithm to operate on the textual data.

```
TF-IDF feature matrix shape: (27476, 10000)
```

Example of raw vs cleaned text:

	text	cleaned_text
0	i'd have responded, if I were going	i d have responded if i were going
1	Sooo SAD I will miss you here in San Diego!!!	sooo sad i will miss you here in san diego
2	my boss is bullying me...	my boss is bullying me
3	what interview! leave me alone	what interview leave me alone
4	Sons of ****, why couldn't they put them on the releases we already bought	sons of why couldn t they put them on the releases we already bought
5	http://www.dothebouncy.com/smf - some shameless plugging for the best Rangers forum on earth	some shameless plugging for the best rangers forum on earth
6	2am feedings for the baby are fun when he is all smiles and coos	am feedings for the baby are fun when he is all smiles and coos
7	Soooo high	soooo high
8	Both of you	both of you
9	Journey!? Wow... u just became cooler. hehe... (is that possible!?)	journey wow u just became cooler hehe is that possible

11. LINEAR SVC CLASSIFIER

LinearSVC is a linear Support Vector Machine classifier provided by Scikit-learn. It is suitable for high-dimensional sparse feature spaces such as TF-IDF text vectors. The classifier learns decision boundaries that separate the sentiment classes. After training, the model assigns the most appropriate sentiment class to each test tweet.

The choice of LinearSVC is appropriate for a text-classification baseline because TF-IDF usually produces a large sparse matrix and linear models can process such representations efficiently.

12. TRAINING AND TESTING

The dataset is divided using an 80:20 train-test split. Stratification is used so that the relative distribution of sentiment classes is maintained across the training and testing subsets. The model learns only from the training data, while the test data is reserved for evaluating generalization to unseen tweets.

```
Training set size: 21980
Test set size: 5496
```

13. SYSTEM WORKFLOW

1. Input Twitter dataset
2. Text preprocessing

3. Split data into training and testing sets
4. TF-IDF vectorization
5. Train LinearSVC classifier
6. Predict sentiment on test tweets
7. Evaluate predictions
8. Display classification metrics and confusion matrix

14. IMPLEMENTATION

The implementation is carried out in Python using a Jupyter Notebook. Pandas is used for dataset handling, Scikit-learn provides the train-test split, TF-IDF vectorization, LinearSVC model, and evaluation metrics, and visualization tools are used to inspect model performance.

The core implementation concept is summarized below:

```
# Main machine-learning workflow
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

model = LinearSVC()
model.fit(X_train_tfidf, y_train)

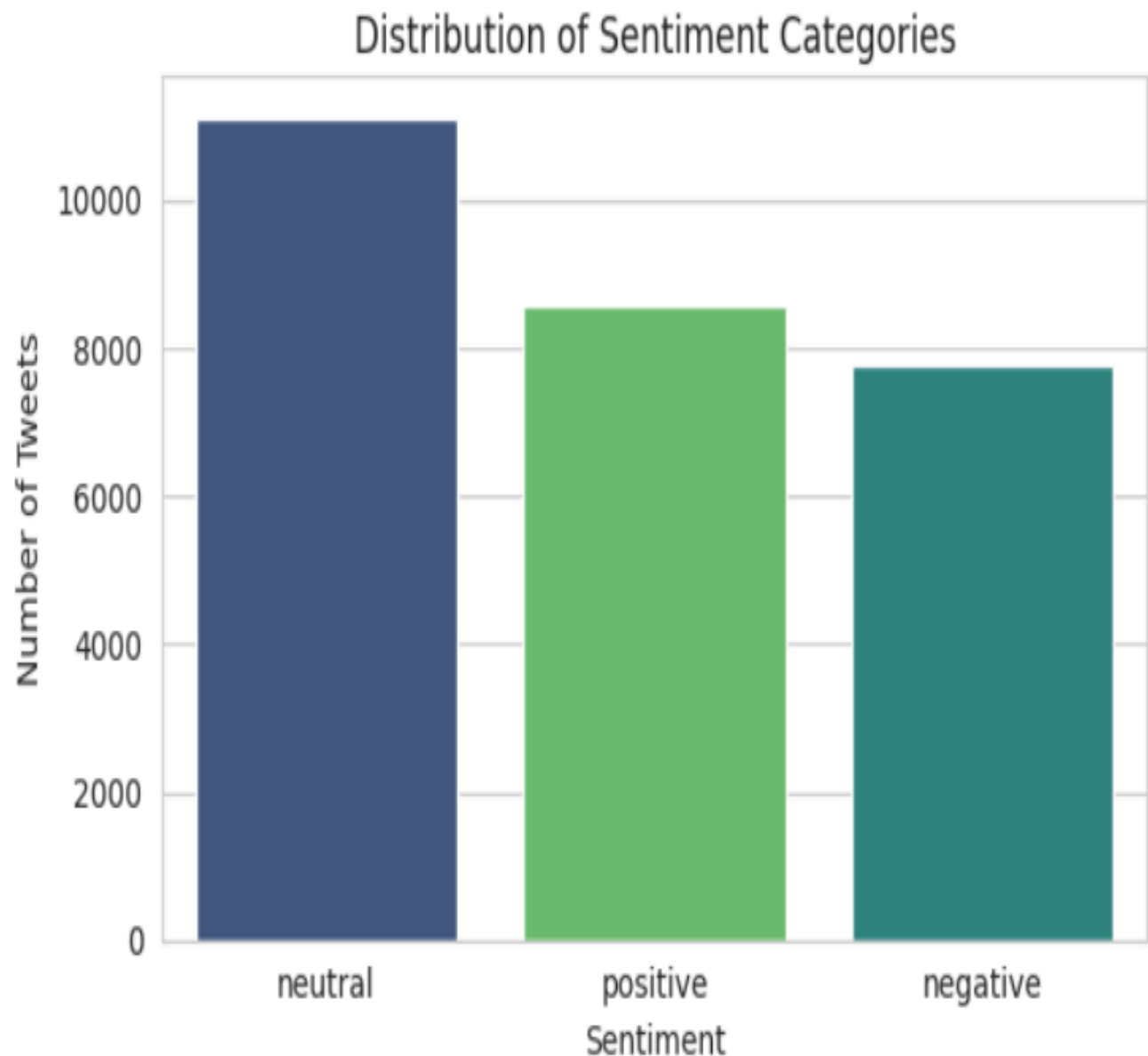
y_pred = model.predict(X_test_tfidf)
```

15. RESULTS AND PERFORMANCE ANALYSIS

The trained LinearSVC model produced the following overall evaluation results on the test set.

Metric	Score
Accuracy	65.99%
Precision	66.38%
Recall	65.99%
F1-score	66.07%

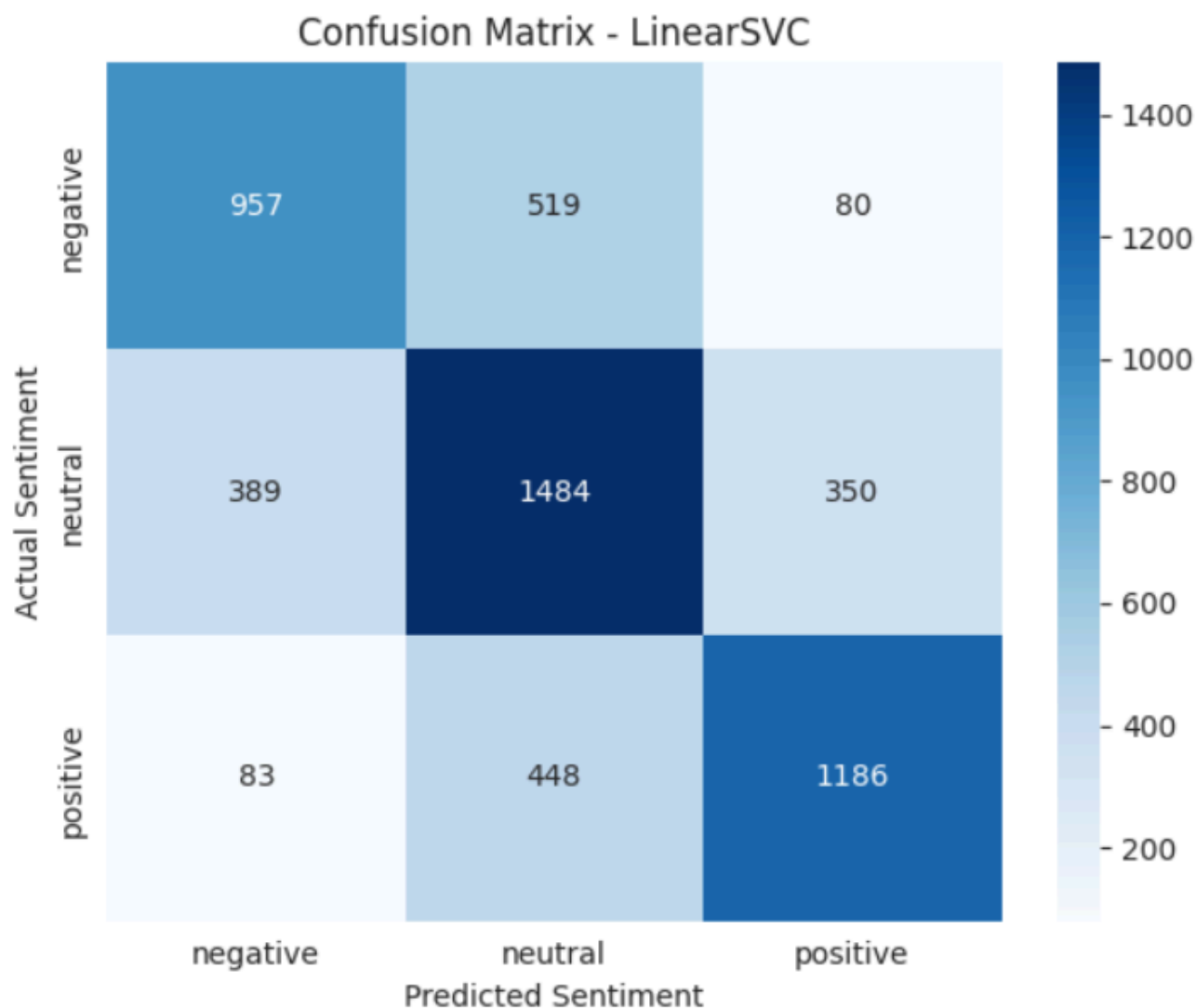
An accuracy of 65.99% means that approximately two-thirds of the test predictions were correctly classified. Precision of 66.38% indicates the overall correctness of positive class assignments when considered across the multi-class evaluation. Recall of 65.99% represents the model's ability to identify the correct instances across the classes. The F1-score of 66.07% provides a balanced measure of precision and recall.



16. CONFUSION MATRIX ANALYSIS

The confusion matrix compares the actual sentiment labels with the labels predicted by the classifier. The diagonal elements represent correctly classified tweets, while off-diagonal elements represent classification errors between sentiment categories. The matrix helps identify which sentiment classes are more frequently confused with one another.

The confusion matrix generated in the notebook should be inserted in this section as a screenshot for the final submission.



17. SAMPLE PREDICTIONS

The trained model can be used to predict the sentiment of new, previously unseen tweets. Each new tweet must undergo the same preprocessing and TF-IDF transformation used during training. The resulting vector is supplied to the trained LinearSVC model, which returns a sentiment category.

The notebook's sample prediction outputs can be inserted here as screenshots.

```
Input: I really love this product. It is amazing!
Predicted Sentiment: positive
-----
Input: This is terrible and I completely hate it.
Predicted Sentiment: negative
-----
Input: I think the meeting is scheduled for tomorrow morning.
Predicted Sentiment: neutral
-----
```

18. ADVANTAGES

- Automates sentiment classification.
- Can process large numbers of tweets faster than manual analysis.
- TF-IDF provides a simple and interpretable text representation.
- LinearSVC is effective for sparse, high-dimensional text features.
- The workflow is relatively lightweight and suitable for a baseline NLP project.

19. LIMITATIONS

- The model may struggle with sarcasm, irony, slang, and context-dependent language.
- Tweets containing multiple sentiments can be difficult to classify into a single class.
- Performance depends strongly on the quality and representativeness of the training dataset.
- TF-IDF does not fully capture word order and deeper semantic relationships.
- The reported performance is a baseline and may be improved with advanced NLP approaches.

20. FUTURE ENHANCEMENTS

- Use word and character n-grams to capture additional textual patterns.
- Experiment with Logistic Regression, Naive Bayes, Random Forest, or other classifiers.
- Apply hyperparameter tuning and cross-validation.
- Use word embeddings or transformer-based models such as BERT for richer semantic understanding.
- Add real-time Twitter/X data collection where permitted by the platform's API and terms.
- Deploy the trained model as a web application or REST API.